

Universe Reinsurance Platform — Database Link & Alignment Checklist

Run after every deploy or repository switch. Extracted from the *Repository Migration & Database Alignment Guide* v1.0, §7 — the guide holds the full context, seeding rules and troubleshooting.

Version 1.0 — September 2026 — Confidential, for internal IT use only. Server layout per the v2.0 specification: `/opt/universe`, user `universe`, `/opt/universe/.env`, PM2, nginx, PostgreSQL on localhost. Source: https://github.com/iwadaya/Saudi_re.

Checklist — is the database linked to the app, and does it align?

Work through the groups in order. "Command" runs on the application server; `psql` commands need `DATABASE_URL` in the shell — load it once with `cd /opt/universe && set -a && . /.env && set +a` (as `universe` or root). Tick every row before sign-off.

7.1 Configuration — the app is told about exactly one database

#	Check	Command	Expected	Proves
C1	One env file, locked down	<code>ls -l /opt/universe/.env</code>	<code>-rw----- universe universe</code>	Secrets readable by the app user only
C2	Exactly one <code>DATABASE_URL</code>	<code>grep -c '^DATABASE_URL=' /opt/universe/.env</code>	1	No duplicate/overridden connection string
C3	No placeholders	<code>grep -c CHANGE_ME /opt/universe/.env</code>	0	Real values everywhere
C4	Production mode, migrations explicit	<code>grep -E '(NODE_ENV RUN_MIGRATIONS_ON_BOOT)=' /opt/universe/.env</code>	<code>NODE_ENV=production, RUN_MIGRATIONS_ON_BOOT=false</code> (or absent)	Schema changes only happen in the deploy step
C5	Seeding posture	<code>grep -E '^SEED_ON_DEPLOY=' /opt/universe/.env</code>	absent, or <code>SEED_ON_DEPLOY=reference</code> — never <code>1 / reset</code>	No fabricated treaties can be seeded
C6	The app loaded <i>that</i> file	<code>sudo -u universe -H pm2 logs universe -l 300 --nostream grep -m1 'startup configuration loaded'</code>	JSON line with <code>"envFile":"/opt/universe/.env"</code>	The running process read this env file, not another

7.2 Connectivity — the database answers with the app's own credentials

#	Check	Command	Expected	Proves
D1	Connect with the app's URL	<code>psql "\$DATABASE_URL" -c 'select 1'</code>	one row, 1	Host, port, role, password and database name are right

#	Check	Command	Expected	Proves
D2	Identify what you are connected to	<code>psql "\$DATABASE_URL" -tAc "select current_database(), current_user, inet_server_addr(), inet_server_port(), version()"</code>	<code>universe universe 127.0.0.1 5432 PostgreSQL 16...</code> (your values)	You are looking at the intended database, not a stray one
D3	Postgres is up and listening locally	<code>sudo systemctl status postgresql --no-pager head -3</code> and <code>ss -tlnp grep 5432</code>	<code>active (running); 127.0.0.1:5432 only</code>	Database service healthy and not exposed
D4	Role owns the schema	<code>psql "\$DATABASE_URL" -tAc "select pg_get_userbyid(datdba) from pg_database where datname=current_database()"</code>	<code>universe</code>	Migrations can create/alter objects without superuser help

7.3 Runtime — the *running* application is connected to *that* database

#	Check	Command	Expected	Proves
R1	Boot log names the database	<code>sudo -u universe -H pm2 logs universe -l -lines 300 --nostream grep -m1 'DB pool connecting'</code>	<code>"url":"postgresql://universe:***@localhost:5432/universe"</code> matching <code>DATABASE_URL</code> (password masked)	The process is using the same host/port/db you tested in D1–D2
R2	Boot sequence completed	same log, <code>grep -E 'database connection verified migrations skipped migrations complete reference data ready http server listening'</code>	All four stages present for the current start: <code>migrations skipped</code> is correct (the deploy applied them; you see <code>migrations complete</code> instead only if <code>RUN_MIGRATIONS_ON_BOOT=true</code>)	App verified the DB, found the schema, seeded, and is serving
R3	Deep health via the app port	<code>curl -sS -D - http://127.0.0.1:4000/api/health/deep</code>	HTTP 200, <code>"db":{"ok":true,"pingMs":<small>,"pool":{"max":50,...}}</code> , header <code>X-Pool-Waiting: 0</code>	Live round-trip from the app to the DB, pool not saturated
R4	Deep health through nginx/TLS	<code>curl -sS https://<APP_DOMAIN>/api/health/deep</code>	same body, <code>"env":"production"</code>	Users' path reaches the same connected app
R5	DB sees the app's connections	<code>psql "\$DATABASE_URL" -c "select username, client_addr, state, count(*) from pg_stat_activity where datname=current_database() group by 1,2,3"</code>	rows with <code>username = universe</code> (idle/active) from the app host	The connections exist in Postgres, from the expected role and host

#	Check	Command	Expected	Proves
R6	Read path: app data = DB data	<pre>curl -sS http://127.0.0.1:4000/api/auth/users grep -o '"username": "[^"]*"' cut - d'"' -f4 sort vs psql "\$DATABASE_URL" -tAc "select username from public.uw_user where is_active order by 1"</pre>	identical lists (and not the two-user demo fallback cuo / underwriter)	The login screen is served from this database
R7	Write path: a login lands in the audit log	<pre>Log in once in the browser, then psql "\$DATABASE_URL" -c "select event_type, actor, created_at from public.audit_log where event_type='LOGIN' order by created_at desc limit 3"</pre>	top row = your username, created_at = just now	The app writes to this database, and the clock/timezone are sane
R8	Smoke test	<pre>BASE_URL=https://<APP_DOMAIN> /opt/universe/deploy/smoke-test.sh (add SMOKE_USER / SMOKE_PASSWORD to test a real login)</pre>	PASSED	All of R3–R6 in one exit code

7.4 Schema alignment — the code and the database agree on the migrations

#	Check	Command	Expected	Proves
S1	Nothing pending	<pre>cd /opt/universe && sudo -u universe -H npm run migrate:status --prefix server tail -n 3</pre>	Pending (0):	Every migration file in this checkout has been applied
S2	Counts match	<pre>psql "\$DATABASE_URL" -tAc "select count(*) from public._migrations" vs ls /opt/universe/server/src/db/migrations/ *.sql wc -l</pre>	equal (149 at this commit)	No file applied twice, none missing
S3	No orphans (DB ahead of code)	<pre>psql "\$DATABASE_URL" -tAc "select filename from public._migrations" sort > /tmp/applied.txt; ls /opt/universe/server/src/db/migrations sort > /tmp/ondisk.txt; comm -23 /tmp/applied.txt /tmp/ondisk.txt</pre>	empty	The database was never migrated by code this checkout does not contain. If a filename prints, the old repository shipped a migration the new one lacks — stop and reconcile before deploying further (docs/migration-audit.md)
S4	No stragglers (code ahead of DB)	<pre>comm -13 /tmp/applied.txt /tmp/ondisk.txt</pre>	empty	Same as S1, file-by-file
S5	Last migration is recent and expected	<pre>psql "\$DATABASE_URL" -c "select filename, applied_at from public._migrations order by applied_at desc limit 5"</pre>	newest = 157_quote_offer_sta tus_checks.sql at this commit, applied_at = your deploy time	The deploy's migration step ran against this database

#	Check	Command	Expected	Proves
S6	Core tables answer	<code>psql "\$DATABASE_URL" -c "select (select count(*) from public.contract) contracts, (select count(*) from public.quote) quotes, (select count(*) from public.claim) claims, (select count(*) from public.uw_user) users"</code>	your real business counts, unchanged from before the switch	The data survived; the app's tables exist

7.5 Reference-data alignment — the lookups the app expects are present

#	Check	Command	Expected	Proves
F1	Reference counts	<code>cd /opt/universe && sudo -u universe -H npm run seed:reference:check</code>	the §5.2 block: countries 75, currencies ≥32, brokers 11, reinsurers 25, cedants ≥40, treaty types 11, classes 15, roles 9; business rows = yours	The reference seed reached this database
F2	Saudi/GCC cedants visible	SQL in §5.2	Tawuniya, MEDGULF, ... listed	GCC extras present
F3	No fabricated treaties	<code>psql "\$DATABASE_URL" -tAc "select count(*) from public.contract where import_metadata->>'source'='seed:test-treaties'"</code>	0	The test portfolio was never loaded
F4	UI dropdowns	New treaty screen: Cedant, Currency, Reinsurer, Class of business, Treaty type	populated as in §5.2	The app reads the seeded lookups

7.6 Capacity and users

#	Check	Command	Expected	Proves
P1	Connection budget	<code>psql "\$DATABASE_URL" -tAc "show max_connections" ; PM2 workers from pm2 status ; per-worker pool from R3 pool.max ; grep -E '^DB_POOL_MAX=' /opt/universe/.env</code>	workers × pool.max ≤ max_connections – 20 (e.g. 4 × 20 = 80 ≤ 80). ecosystem.config.cjs derives 20 per worker on a 4-core box only while DB_POOL_MAX is not set in .env — a DB_POOL_MAX=50 line (as in older notes) is used verbatim per worker: 4 × 50 = 200 > 100. Remove it or set it to the per-worker value	The cluster cannot exhaust Postgres

#	Check	Command	Expected	Proves
P2	Accounts	<code>cd /opt/universe/server && sudo -u universe -H node scripts/manage-user.js list</code>	your real users active; seeded personas (<code>chief.underwriter</code> , <code>retro.manager</code> , <code>underwriter1-4</code>) rotated or deactivated	No <code>demo2026</code> login remains
P3	Backup after the switch	<code>ls -lt /var/backups/universe head -2 ; weekly scripts/verify-restore.sh exit 0</code>	a dump newer than the deploy	Recovery point reflects the migrated schema

7.7 Sign-off summary

- ❑ §7.1 C1–C6: one env file, one `DATABASE_URL` , production mode, seeding posture correct, app loaded that file.
- ❑ §7.2 D1–D4: `psql` with the app's URL works and identifies the intended database; role owns it.
- ❑ §7.3 R1–R8: boot log, deep health (direct and via nginx), `pg_stat_activity` , user list = `uw_user` , login audited, smoke test `PASSED` .
- ❑ §7.4 S1–S6: `Pending (0)` , counts equal, no orphans, no stragglers, last migration as expected, business counts unchanged.
- ❑ §7.5 F1–F4: reference counts as in §5.2, GCC cedants visible, zero fabricated treaties, dropdowns populated.
- ❑ §7.6 P1–P3: connection budget, accounts rotated, fresh backup.

Reference — expected seed counts on a freshly migrated database

``` Reference data after seeding: countries 75 currencies 32 brokers 11 reinsurers 25 cedants 40 treaty\_types 11 classes\_of\_business 15 roles 9 Business data (unchanged): contracts 0 quotes 0 claims 0 users 6

On an existing database expect at least these reference counts (plus the Ghana pack and user-added rows); the business rows must be your real, unchanged numbers.

## Sign-off

| Item                                        | Value                                                                                  |
|---------------------------------------------|----------------------------------------------------------------------------------------|
| Date / time                                 |                                                                                        |
| Performed by                                |                                                                                        |
| Deployed commit ( <code>git log -1</code> ) |                                                                                        |
| <code>migrate:status</code> result          | Pending (0) <input type="checkbox"/>                                                   |
| Smoke test (R8)                             | PASSED <input type="checkbox"/>                                                        |
| Orphan / straggler check (S3, S4)           | both empty <input type="checkbox"/>                                                    |
| Reference counts (F1)                       | countries ..... currencies ..... brokers ..... reinsurers ..... cedants .....<br>..... |
| Business counts (S6)                        | contracts ..... quotes ..... claims ..... users .....                                  |